Performance Analysis and Comparision of Classification Algorithms

(Logistic Regression, KNN, SVM, Decision Tree) on Palmet Penguins dataset

TEAM-04

Shikha R. Upadhyay- 23911A3557

Pranitha- 23911A3517

Gowtham Mudiraj- 23911A3502

Shashi Vardhan- 23911A3537

PROBLEM STATEMENT :

The Palmer Penguins dataset provides physical measurements for three penguin species: Adelie, Chinstrap, and Gentoo, collected from islands in Antarctica. Each entry includes features like:

Bill length and depth

Flipper length

Body mass

Gender and island location

The goal is to build a machine learning model that can accurately predict a penguin's species based on these features.

STEPS:

1. Understanding the problem statement.
2. Identify the dataset.
3. Import required library files.
4. Load the dataset in the environment.
5. Data preprocessing.
6. Data splitting.
7. Build the model.
8. Train the model.
9. Test the model.
10. Check the performance of the model.

Logistic Regression -

Logistic Regression predicts the probability that a given input belongs to a certain class. It then uses that probability to classify the input.

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

1. Importing the library files

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import linear_model
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

NumPy - A Python library for numerical computations and handling large, multi-dimensional arrays and matrices.

Pandas - A data manipulation and analysis library in Python, offering data structures like DataFrame for structured data.

Matplotlib - A Python library used for creating static, interactive, and animated visualizations.

Seaborn - A Python data visualization library built on Matplotlib that provides a high-level interface for drawing attractive and informative statistical graphics.

Scikit-learn - A machine learning library in Python that provides tools for data mining, analysis, and modeling.

2. Loading the Dataset

```
dataset = pd.read_csv('/content/drive/My Drive/penguins_size.csv')
```

```
dataset
```

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	MALE
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	FEMALE
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	FEMALE
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	FEMALE
...	...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	FEMALE
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	MALE
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	FEMALE
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0	MALE

344 rows × 7 columns

Next steps: Generate code with `dataset` View recommended plots New interactive sheet

```
#Display the size of the dataset
print(dataset.shape)
```

```
(344, 7)
```

```
#Display the first five rows of the dataset
print(dataset.head())
```

```
   species     island  culmen_length_mm  culmen_depth_mm  flipper_length_mm  body_mass_g     sex
0  Adelie  Torgersen              39.1             18.7              181.0       3750.0    MALE
1  Adelie  Torgersen              39.5             17.4              186.0       3800.0  FEMALE
2  Adelie  Torgersen              40.3             18.0              195.0       3250.0  FEMALE
3  Adelie  Torgersen               NaN              NaN                NaN          NaN     NaN
4  Adelie  Torgersen              36.7             19.3              193.0       3450.0  FEMALE
```

```
X = dataset.iloc[:,0:4]
y = dataset.iloc[:,-1]
```

```
X
```

	species	island	culmen_length_mm	culmen_depth_mm
0	Adelie	Torgersen	39.1	18.7
1	Adelie	Torgersen	39.5	17.4
2	Adelie	Torgersen	40.3	18.0
3	Adelie	Torgersen	NaN	NaN
4	Adelie	Torgersen	36.7	19.3
...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3
341	Gentoo	Biscoe	50.4	15.7
342	Gentoo	Biscoe	45.2	14.8
343	Gentoo	Biscoe	49.9	16.1

344 rows × 4 columns

Next steps: Generate code with X | View recommended plots | New interactive sheet

y

	sex
0	MALE
1	FEMALE
2	FEMALE
3	NaN
4	FEMALE
...	...
339	NaN
340	FEMALE
341	MALE
342	FEMALE
343	MALE

344 rows × 1 columns

dtype: object

3. Preprocessing

```
X = dataset[['culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm', 'body_mass_g']].values # Select numerical features for X
y = dataset['species'].values # Select the 'species' column for y

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X = sc.fit_transform(X) # Now X contains only numerical features and can be scaled
X
```

```
array([[-0.88449874,  0.78544923, -1.41834665, -0.56414208],
       [-0.81112573,  0.1261879 , -1.06225022, -0.50170305],
       [-0.66437972,  0.43046236, -0.42127665, -1.18853234],
       ...,
       [ 1.18828874, -0.73592307,  1.50164406,  1.93341896],
       [ 0.23443963, -1.19233476,  0.7894512 ,  1.24658968],
       [ 1.09657248, -0.53307343,  0.86067049,  1.49634578]])
```

4. Split the dataset into Training and Testing

```
# Impute missing values with the mean for numerical features
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')

# Fit the imputer on your numerical features only
numerical_features = ['culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm', 'body_mass_g']
dataset[numerical_features] = imputer.fit_transform(dataset[numerical_features])

# Now, proceed with feature selection and scaling
X = dataset[numerical_features].values
y = dataset['species'].values

# Split the data into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=16)
```

```
X_train
```

```
array([[  47.5,   14. ,  212. , 4875. ],
       [  49. ,   16.1,  216. , 5550. ],
       [  49.1,   15. ,  228. , 5500. ],
       [  48.1,   15.1,  209. , 5500. ],
       [  37.2,   18.1,  178. , 3900. ],
       [  48.7,   14.1,  210. , 4450. ],
       [  47.4,   14.6,  212. , 4725. ],
       [  34.5,   18.1,  187. , 2900. ],
       [  40.9,   13.7,  214. , 4650. ],
       [  50.5,   19.6,  201. , 4050. ],
       [  49. ,   19.5,  210. , 3950. ],
       [  53.4,   15.8,  219. , 5500. ],
       [  46.5,   14.8,  217. , 5200. ],
```

```
[  50.2,   18.7,  198. , 3775. ],
[  39.2,   18.6,  190. , 4250. ],
[  43.5,   14.2,  220. , 4700. ],
[  45.2,   16.6,  191. , 3250. ],
[  47.8,   15. ,  215. , 5650. ],
[  40.6,   18.6,  183. , 3550. ],
[  38.7,   19. ,  195. , 3450. ],
[  36.5,   16.6,  181. , 2850. ],
[  35.1,   19.4,  193. , 4200. ],
[  36.6,   17.8,  185. , 3700. ],
[  49.1,   14.8,  220. , 5150. ],
[  49.8,   15.9,  229. , 5950. ],
[  46.8,   14.3,  215. , 4850. ],
[  42.5,   16.7,  187. , 3350. ],
[  37.3,   17.8,  191. , 3350. ],
[  50. ,   15.3,  220. , 5550. ],
[  52.2,   17.1,  228. , 5400. ],
[  41.1,   17.5,  190. , 3900. ],
[  50.5,   15.9,  222. , 5550. ],
[  45.3,   13.8,  208. , 4200. ],
[  55.1,   16. ,  230. , 5850. ],
[  50.9,   17.9,  196. , 3675. ],
[  37.6,   19.3,  181. , 3300. ],
[  46.4,   17.8,  191. , 3700. ],
[  46.4,   15.6,  221. , 5000. ],
[  51. ,   18.8,  203. , 4100. ],
[  51.3,   18.2,  197. , 3750. ],
[  41.7,   14.7,  210. , 4700. ],
[  46.7,   17.9,  195. , 3300. ],
[  59.6,   17. ,  230. , 6050. ],
[  39.5,   16.7,  178. , 3250. ],
[  48.5,   17.5,  191. , 3400. ],
[  45.5,   17. ,  196. , 3500. ],
[  38.1,   18.6,  190. , 3700. ],
[  39.6,   18.8,  190. , 4600. ],
[  52.5,   15.6,  221. , 5450. ],
[  41.1,   19.1,  188. , 4100. ],
[  35.9,   19.2,  189. , 3800. ],
[  37. ,   16.9,  185. , 3000. ],
[  40.8,   18.4,  195. , 3900. ],
[  49.5,   19. ,  200. , 3800. ],
[  43.2,   14.5,  208. , 4450. ],
[  38.8,   17.2,  180. , 3800. ],
[  50.7,   19.7,  203. , 4050. ],
[  42.1,   19.1,  195. , 4000. ],
```

```
y_train
```

```
array(['Gentoo', 'Gentoo', 'Gentoo', 'Gentoo', 'Adelie', 'Gentoo',
       'Gentoo', 'Adelie', 'Gentoo', 'Chinstrap', 'Chinstrap', 'Gentoo',
       'Gentoo', 'Chinstrap', 'Adelie', 'Gentoo', 'Chinstrap', 'Gentoo',
       'Adelie', 'Adelie', 'Adelie', 'Adelie', 'Adelie', 'Gentoo',
       'Gentoo', 'Gentoo', 'Chinstrap', 'Adelie', 'Gentoo', 'Gentoo',
       'Adelie', 'Gentoo', 'Gentoo', 'Gentoo', 'Chinstrap', 'Adelie',
       'Chinstrap', 'Gentoo', 'Chinstrap', 'Chinstrap', 'Gentoo',
       'Chinstrap', 'Gentoo', 'Adelie', 'Chinstrap', 'Chinstrap',
       'Adelie', 'Adelie', 'Gentoo', 'Adelie', 'Adelie', 'Adelie',
       'Adelie', 'Chinstrap', 'Gentoo', 'Adelie', 'Chinstrap', 'Adelie',
       'Gentoo', 'Adelie', 'Adelie', 'Adelie', 'Adelie', 'Adelie',
       'Adelie', 'Adelie', 'Adelie', 'Gentoo', 'Chinstrap', 'Chinstrap',
       'Chinstrap', 'Adelie', 'Adelie', 'Gentoo', 'Chinstrap', 'Adelie',
       'Chinstrap', 'Gentoo', 'Adelie', 'Chinstrap', 'Adelie',
       'Chinstrap', 'Adelie', 'Adelie', 'Chinstrap', 'Gentoo', 'Adelie',
       'Gentoo', 'Chinstrap', 'Chinstrap', 'Gentoo', 'Adelie',
       'Chinstrap', 'Gentoo', 'Adelie', 'Gentoo', 'Adelie', 'Chinstrap',
       'Adelie', 'Chinstrap', 'Gentoo', 'Gentoo', 'Gentoo', 'Adelie',
       'Chinstrap', 'Adelie', 'Adelie', 'Gentoo', 'Adelie', 'Adelie',
       'Adelie', 'Gentoo', 'Adelie', 'Adelie', 'Gentoo', 'Gentoo',
       'Gentoo', 'Gentoo', 'Gentoo', 'Chinstrap', 'Adelie', 'Adelie',
       'Gentoo', 'Adelie', 'Gentoo', 'Adelie', 'Adelie', 'Adelie',
       'Adelie', 'Gentoo', 'Adelie', 'Gentoo', 'Adelie', 'Adelie',
       'Gentoo', 'Chinstrap', 'Gentoo', 'Adelie', 'Gentoo', 'Chinstrap',
       'Gentoo', 'Adelie', 'Adelie', 'Gentoo', 'Adelie', 'Gentoo',
       'Adelie', 'Chinstrap', 'Adelie', 'Chinstrap', 'Adelie', 'Adelie',
       'Gentoo', 'Gentoo', 'Adelie', 'Chinstrap', 'Gentoo', 'Gentoo',
       'Adelie', 'Gentoo', 'Chinstrap', 'Gentoo', 'Gentoo', 'Adelie',
       'Gentoo', 'Chinstrap', 'Adelie', 'Gentoo', 'Chinstrap',
       'Chinstrap', 'Gentoo', 'Adelie', 'Adelie', 'Chinstrap', 'Adelie',
       'Adelie', 'Gentoo', 'Gentoo', 'Adelie', 'Adelie', 'Adelie',
       'Adelie', 'Gentoo', 'Adelie', 'Adelie', 'Chinstrap', 'Chinstrap',
       'Gentoo', 'Gentoo', 'Adelie', 'Gentoo', 'Adelie', 'Adelie',
       'Adelie', 'Gentoo', 'Adelie', 'Adelie', 'Adelie', 'Chinstrap',
       'Gentoo', 'Gentoo', 'Gentoo', 'Gentoo', 'Adelie', 'Adelie',
       'Chinstrap', 'Chinstrap', 'Chinstrap', 'Chinstrap', 'Gentoo',
       'Gentoo', 'Gentoo', 'Gentoo', 'Adelie', 'Gentoo', 'Gentoo',
       'Gentoo', 'Adelie', 'Adelie', 'Adelie', 'Chinstrap', 'Gentoo',
       'Adelie', 'Gentoo', 'Gentoo', 'Chinstrap', 'Adelie', 'Gentoo',
       'Adelie', 'Gentoo', 'Gentoo', 'Adelie', 'Chinstrap', 'Adelie',
       'Chinstrap', 'Adelie', 'Gentoo', 'Adelie', 'Chinstrap', 'Adelie',
```

```
       'Gentoo', 'Chinstrap', 'Gentoo', 'Gentoo', 'Adelie', 'Gentoo',
       'Adelie', 'Adelie', 'Gentoo', 'Chinstrap'], dtype=object)
```

```
X_train
```

```
array([[  35.6,   17.5,  191. , 3175. ],
       [  39.3,   20.6,  190. , 3650. ],
       [  49.8,   16.8,  230. , 5700. ],
       [  37.9,   18.6,  172. , 3150. ],
       [  35.9,   16.6,  190. , 3050. ],
       [  40.5,   18.9,  180. , 3950. ],
       [  49.8,   17.3,  198. , 3675. ],
       [  45. ,   15.4,  220. , 5050. ],
       [  44.5,   14.7,  214. , 4850. ],
       [  45.2,   15.8,  215. , 5300. ],
       [  44. ,   13.6,  208. , 4350. ],
       [  40.6,   19. ,  199. , 4000. ],
       [  42.2,   19.5,  197. , 4275. ],
       [  49. ,   19.6,  212. , 4300. ],
       [  46.5,   17.9,  192. , 3500. ],
       [  53.5,   19.9,  205. , 4500. ],
       [  51.3,   19.2,  193. , 3650. ],
       [  46.2,   14.9,  221. , 5300. ],
       [  45.8,   14.2,  219. , 4700. ],
       [  50.4,   15.3,  224. , 5550. ],
       [  45.2,   14.8,  212. , 5200. ],
       [  37.8,   18.3,  174. , 3400. ],
       [  49.2,   15.2,  221. , 6300. ],
       [  46.1,   15.1,  215. , 5100. ],
       [  46.8,   15.4,  215. , 5150. ],
       [  36. ,   18.5,  186. , 3100. ],
       [  46. ,   21.5,  194. , 4200. ],
       [  37.5,   18.5,  199. , 4475. ],
       [  49.7,   18.6,  195. , 3600. ],
       [  44.9,   13.8,  212. , 4750. ],
       [  41.5,   18.5,  201. , 4000. ],
       [  45.8,   14.6,  210. , 4200. ],
       [  48.7,   15.1,  222. , 5350. ],
       [  49.2,   18.2,  195. , 4400. ],
       [  38.6,   21.2,  191. , 3800. ],
       [  46.5,   14.5,  213. , 4400. ],
       [  36.2,   17.2,  187. , 3150. ],
       [  51.5,   16.3,  230. , 5500. ],
       [  48.2,   15.6,  221. , 5100. ],
       [  38.2,   20. ,  190. , 3900. ],
       [  45.6,   19.4,  194. , 3525. ],
       [  43.2,   18.5,  192. , 4100. ],
       [  51.9,   19.5,  206. , 3950. ],
       [  34.6,   17.2,  189. , 3200. ],
       [  48.6,   16. ,  230. , 5800. ],
       [  39.7,   18.4,  190. , 3900. ],
       [  45.9,   17.1,  190. , 3575. ],
       [  40.2,   17. ,  176. , 3450. ],
       [  47.2,   15.5,  215. , 4975. ],
       [  47. ,   17.3,  185. , 3700. ],
       [  50.8,   17.3,  228. , 5600. ],
       [  50.5,   15.9,  225. , 5400. ],
       [  44.1,   18. ,  210. , 4000. ],
       [  50.5,   15.2,  216. , 5000. ],
       [  42.8,   18.5,  195. , 4250. ],
       [  41.5,   18.3,  195. , 4300. ],
       [  42.9,   13.1,  215. , 5000. ],
       [  50.6,   19.4,  193. , 3800. ]])
```

```
y_test
```

```
array(['Gentoo', 'Adelie', 'Adelie', 'Gentoo', 'Adelie', 'Gentoo',
       'Chinstrap', 'Adelie', 'Adelie', 'Gentoo', 'Adelie', 'Adelie',
       'Chinstrap', 'Gentoo', 'Adelie', 'Chinstrap', 'Chinstrap',
       'Adelie', 'Gentoo', 'Adelie', 'Chinstrap', 'Adelie', 'Gentoo',
       'Adelie', 'Gentoo', 'Gentoo', 'Gentoo', 'Gentoo', 'Adelie',
       'Adelie', 'Gentoo', 'Adelie', 'Adelie', 'Gentoo', 'Gentoo',
       'Chinstrap', 'Adelie', 'Chinstrap', 'Gentoo', 'Adelie', 'Adelie',
       'Adelie', 'Adelie', 'Gentoo', 'Adelie', 'Adelie', 'Chinstrap',
       'Gentoo', 'Adelie', 'Gentoo', 'Adelie', 'Adelie', 'Gentoo',
       'Gentoo', 'Adelie', 'Chinstrap', 'Gentoo', 'Adelie', 'Adelie',
       'Gentoo', 'Gentoo', 'Gentoo', 'Adelie', 'Adelie', 'Adelie',
       'Adelie', 'Adelie', 'Adelie', 'Chinstrap', 'Gentoo', 'Adelie',
       'Chinstrap', 'Adelie', 'Chinstrap', 'Adelie', 'Gentoo', 'Adelie',
       'Chinstrap', 'Chinstrap', 'Adelie', 'Adelie', 'Chinstrap',
       'Adelie', 'Gentoo'], dtype=object)
```

5. Building the model

```
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression(random_state=123)
```

```
logreg.fit(X_train, y_train)
y_pred = logreg.predict(X_test)
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
from sklearn import metrics

cmf_matrix = metrics.confusion_matrix(y_test, y_pred)
cmf_matrix
```

```
array([[40,  2,  0],
       [ 0, 15,  0],
       [ 0,  0, 27]])
```

6. Performance of the model

```
from sklearn.metrics import classification_report
print("\nClassification Report of Logistic Regression:\n",metrics.classification_report(y_test,y_pred))
print ("------------------------------------------------------------------------")
```

```
Classification Report of Logistic Regression:
                precision    recall  f1-score   support

        Adelie       1.00      0.95      0.98        42
     Chinstrap       0.88      1.00      0.94        15
        Gentoo       1.00      1.00      1.00        27

      accuracy                           0.98        84
     macro avg       0.96      0.98      0.97        84
  weighted avg       0.98      0.98      0.98        84

------------------------------------------------------------------------
```

KNN Model : K-Nearest Neighbor

A simple, instance-based learning algorithm that classifies data based on the majority class of its nearest neighbors.

Performance Analysis

```
knn = KNeighborsClassifier(n_neighbors=5)
classifier = knn.fit(X_train, y_train)
y_pred_knn = classifier.predict(X_test)
```

```
i = 0
print ("\n------------------------------------------------------------------------")
print ('%-25s %-25s %-25s' % ('Original Label', 'Predicted Label', 'Correct/Wrong'))
print ("------------------------------------------------------------------------")
for label in y_test:
 print ('%-25s %-25s' % (label, y_pred_knn[i]), end="")
 if (label == y_pred_knn[i]):
  print (' %-25s' % ('Correct'))
 else:
  print (' %-25s' % ('Wrong'))
 i = i + 1
print ("------------------------------------------------------------------------")
```

```
------------------------------------------------------------------------
Original Label            Predicted Label           Correct/Wrong
------------------------------------------------------------------------
Gentoo                    Gentoo                    Correct
Adelie                    Chinstrap                 Wrong
Adelie                    Adelie                    Correct
Gentoo                    Gentoo                    Correct
Adelie                    Chinstrap                 Wrong
Gentoo                    Gentoo                    Correct
Chinstrap                 Chinstrap                 Correct
Adelie                    Adelie                    Correct
```

```
Adelie                  Adelie                  Correct
Gentoo                  Gentoo                  Correct
Adelie                  Adelie                  Correct
Adelie                  Adelie                  Correct
Chinstrap               Adelie                  Wrong
Gentoo                  Gentoo                  Correct
Adelie                  Adelie                  Correct
Chinstrap               Adelie                  Wrong
Chinstrap               Adelie                  Wrong
Adelie                  Adelie                  Correct
Gentoo                  Gentoo                  Correct
Adelie                  Adelie                  Correct
Chinstrap               Chinstrap               Correct
Adelie                  Adelie                  Correct
Gentoo                  Gentoo                  Correct
Adelie                  Adelie                  Correct
Gentoo                  Gentoo                  Correct
Gentoo                  Gentoo                  Correct
Gentoo                  Chinstrap               Wrong
Gentoo                  Adelie                  Wrong
Adelie                  Adelie                  Correct
Adelie                  Adelie                  Correct
Gentoo                  Gentoo                  Correct
Adelie                  Adelie                  Correct
Adelie                  Adelie                  Correct
Gentoo                  Gentoo                  Correct
Gentoo                  Gentoo                  Correct
Chinstrap               Adelie                  Wrong
Adelie                  Adelie                  Correct
Chinstrap               Adelie                  Wrong
Gentoo                  Gentoo                  Correct
Adelie                  Adelie                  Correct
Adelie                  Chinstrap               Wrong
Adelie                  Adelie                  Correct
Adelie                  Adelie                  Correct
Gentoo                  Adelie                  Wrong
Adelie                  Adelie                  Correct
Adelie                  Adelie                  Correct
Chinstrap               Chinstrap               Correct
Gentoo                  Gentoo                  Correct
Adelie                  Adelie                  Correct
Gentoo                  Gentoo                  Correct
Adelie                  Adelie                  Correct
Adelie                  Adelie                  Correct
Gentoo                  Gentoo                  Correct
Gentoo                  Gentoo                  Correct
```

```
print("\nClassification Report:\n",metrics.classification_report(y_test, y_pred_knn))
print ("------------------------------------------------------------------------")
```

```
Classification Report:
                precision    recall  f1-score   support

       Adelie       0.69      0.81      0.75        42
    Chinstrap       0.36      0.27      0.31        15
       Gentoo       0.92      0.81      0.86        27

     accuracy                           0.71        84
    macro avg       0.66      0.63      0.64        84
weighted avg       0.71      0.71      0.71        84

------------------------------------------------------------------------
```

SVM : SUPPORT VECTOR MACHINE

Supervised learning algorithm that finds the optimal hyperplane to separate data into different classes.

Performance Analysis

```
# Building a Support Vector Machine on train data
svc_model = SVC(C= .1, kernel='linear', gamma= 1)
svc_model.fit(X_train, y_train)
```

```
SVC(C=0.1, gamma=1, kernel='linear')
```

```
y_pred_svc = svc_model.predict(X_test)
```

```
print("Accuracy of the classifier is %0.2f" % metrics.accuracy_score(y_test,y_pred_svc))
print("------------------------------------------------------------------------")
```

```
Accuracy of the classifier is 0.99
----------------------------------------------------------------------
```

```
from sklearn.svm import SVC
# Building a Support Vector Machine on train data
svc_model = SVC(C= 0.8, kernel='linear', gamma= 8)
svc_model.fit(X_train, y_train)
```

```
SVC(C=0.8, gamma=8, kernel='linear')
```

```
i = 0
print ("\n---------------------------------------------------------------------------")
print ('%-25s %-25s %-25s' % ('Original Label', 'Predicted Label', 'Correct/Wrong'))
print ("---------------------------------------------------------------------------")
for label in y_test:
 print ('%-25s %-25s' % (label, y_pred_svc[i]), end="")
if (label == y_pred_svc[i]):
  print (' %-25s' % ('Correct'))
else:
 print (' %-25s' % ('Wrong'))
 i = i + 1
print ("---------------------------------------------------------------------------")
```

```
---------------------------------------------------------------------------
Original Label            Predicted Label           Correct/Wrong
---------------------------------------------------------------------------
Gentoo                    Gentoo                    Adelie                    Gentoo                    Adelie                    Gent
---------------------------------------------------------------------
```

```
print("Accuracy of the classifier is %0.2f" % metrics.accuracy_score(y_test,y_pred_svc))
print("----------------------------------------------------------------------")
```

```
Accuracy of the classifier is 0.99
----------------------------------------------------------------------
```

DECISION TREES

A tree-like model used for classification and regression, where data is split based on feature values.

Performance Analysis

```
#Train the Decision Tree Classifier
clf_train = DecisionTreeClassifier(criterion='gini',random_state=1234)
dt_train = clf_train.fit(X_train, y_train)
```

```
y_pred_dt = dt_train.predict(X_test)
```

Performance of the Model

```
print(dt_train.score(X_train, y_train))
print(dt_train.score(X_test, y_test))
```

```
1.0
0.9404761904761905
```

```
class_wise = metrics.classification_report(y_true=y_test, y_pred=y_pred)
print(class_wise)
```

```
              precision    recall  f1-score   support

      Adelie       1.00      0.95      0.98        42
   Chinstrap       0.88      1.00      0.94        15
      Gentoo       1.00      1.00      1.00        27

    accuracy                           0.98        84
   macro avg       0.96      0.98      0.97        84
weighted avg       0.98      0.98      0.98        84
```

```
print('Accuracy of the Decision Tree classifer is %0.2f' % metrics.accuracy_score(y_test,y_pred_dt))
print ("------------------------------------------------------------------------")
```

```
Accuracy of the Decision Tree classifer is 0.94
------------------------------------------------------------------------
```

Accuracy of the Decision Tree classifier is 0.94

Visualize the Model

Visualize the Decision Tree on Training Data

```
text_representation = tree.export_text(clf_train)
print(text_representation)
```

```
|--- feature_2 <= 207.00
|   |--- feature_0 <= 43.35
|   |   |--- feature_0 <= 42.30
|   |   |   |--- class: Adelie
|   |   |--- feature_0 >  42.30
|   |   |   |--- feature_0 <= 42.60
|   |   |   |   |--- class: Chinstrap
|   |   |   |--- feature_0 >  42.60
|   |   |   |   |--- class: Adelie
|   |--- feature_0 >  43.35
|   |   |--- feature_1 <= 21.15
|   |   |   |--- feature_3 <= 4125.00
|   |   |   |   |--- class: Chinstrap
|   |   |   |--- feature_3 >  4125.00
|   |   |   |   |--- feature_0 <= 45.90
|   |   |   |   |   |--- class: Adelie
|   |   |   |   |--- feature_0 >  45.90
|   |   |   |   |   |--- class: Chinstrap
|   |   |--- feature_1 >  21.15
|   |   |   |--- class: Adelie
|--- feature_2 >  207.00
|   |--- feature_1 <= 17.65
|   |   |--- class: Gentoo
|   |--- feature_1 >  17.65
|   |   |--- feature_0 <= 46.55
|   |   |   |--- class: Adelie
|   |   |--- feature_0 >  46.55
|   |   |   |--- class: Chinstrap
```

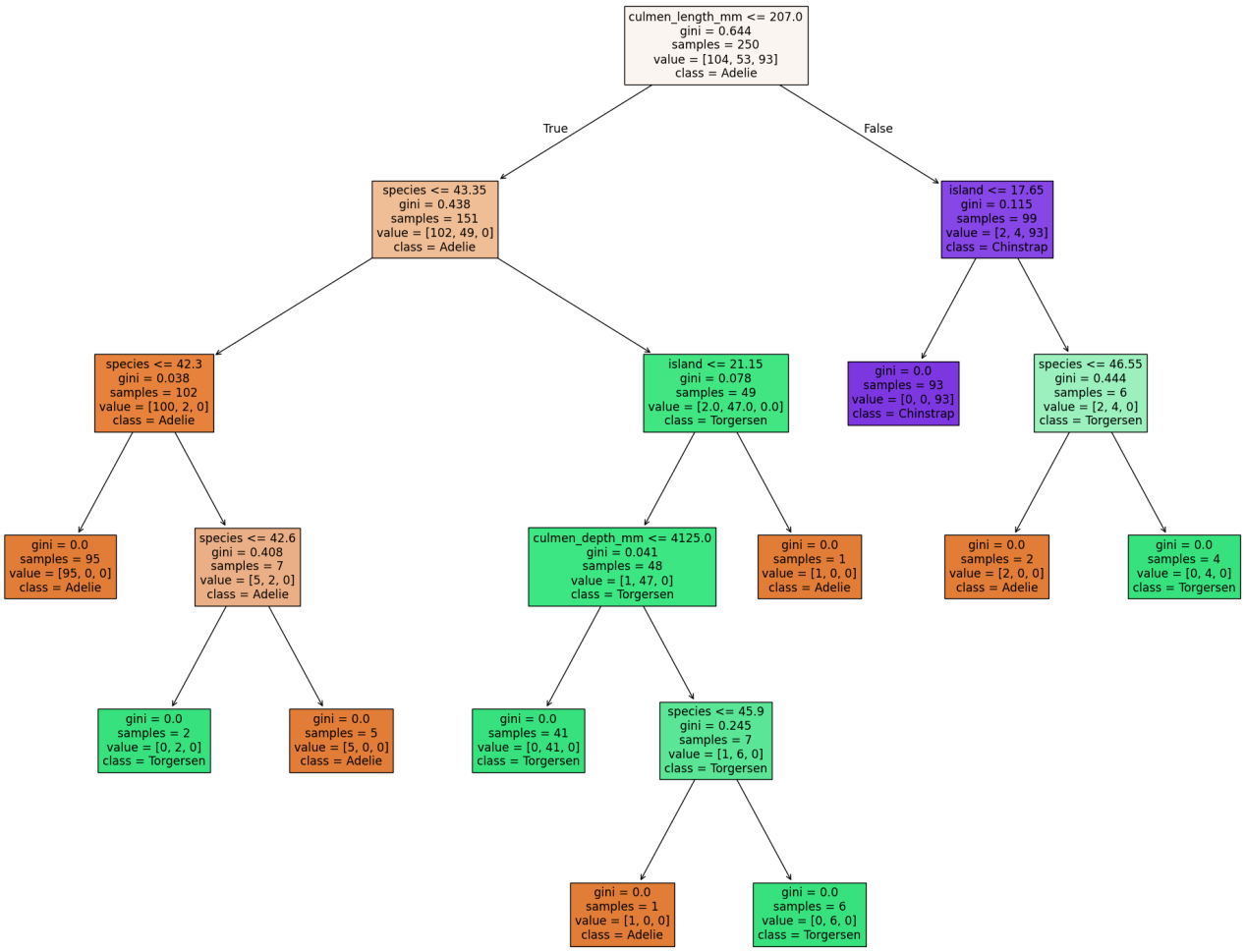
To Generate and display a human-readable text representation of the trained Decision Tree model.

```
with open("decistion_tree_train.log", "w") as fout:
 fout.write(text_representation)
```

To save the textual representation of the trained Decision Tree model to a file

```
fn=['species','island','culmen_length_mm','culmen_depth_mm','flipper_length_mm','body_mass_g']
cn=['Adelie', 'Torgersen','Chinstrap','Dream','Gentoo','Biscoe']
```

```
fig = plt.figure(figsize=(25,20))
tree.plot_tree(clf_train, feature_names=fn, class_names=cn,filled=True)
fig.savefig('imagename.png')
```



Visualize the Decision Tree on Testing Data

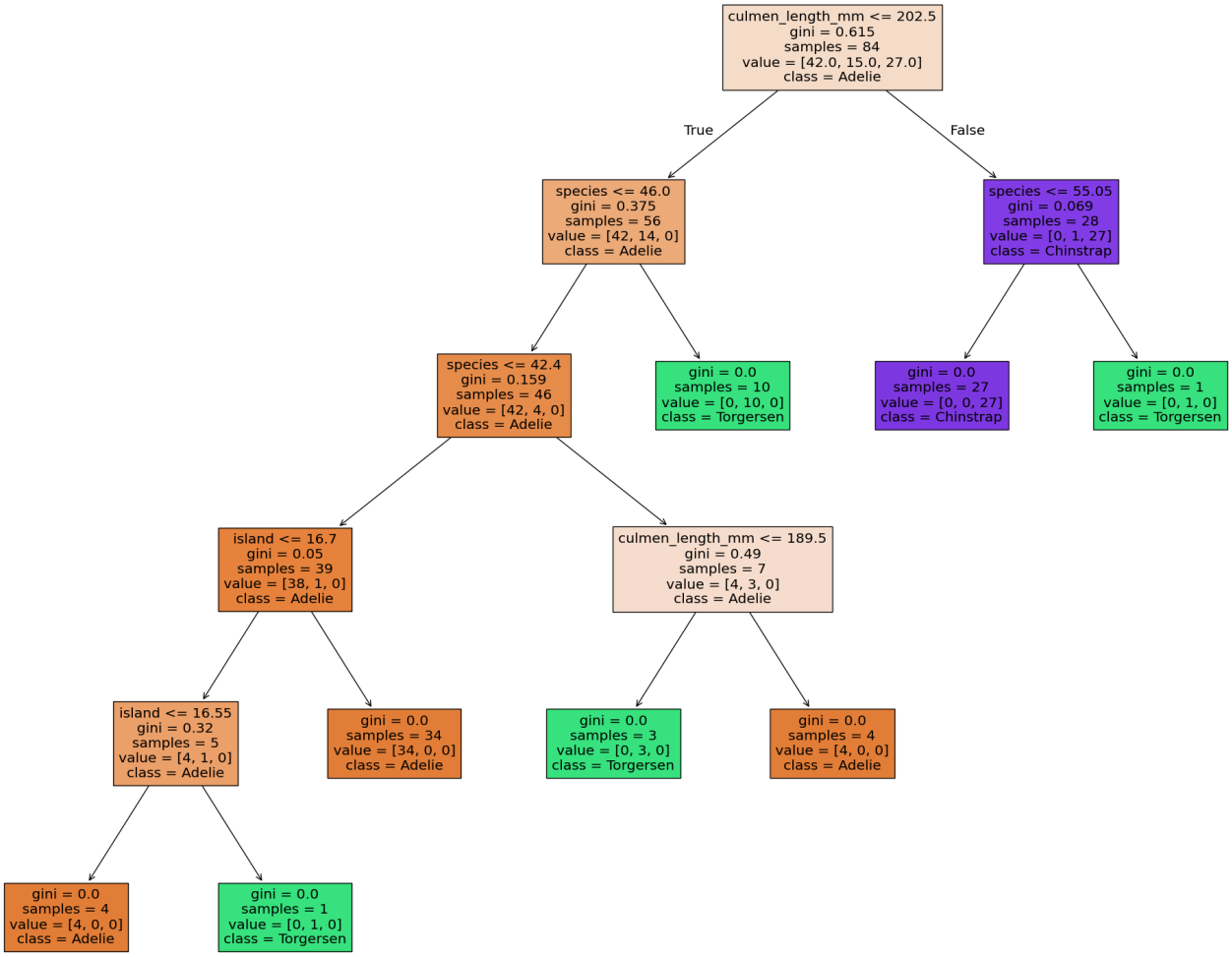
```
clf_test = DecisionTreeClassifier(random_state=1234)
dt_test = clf_test.fit(X_test, y_test)
```

```
text_representation = tree.export_text(clf_test)
print(text_representation)
```

```
|--- feature_2 <= 202.50
|   |--- feature_0 <= 46.00
|   |   |--- feature_0 <= 42.40
|   |   |   |--- feature_1 <= 16.70
|   |   |   |   |--- feature_1 <= 16.55
|   |   |   |   |   |--- class: Adelie
|   |   |   |   |--- feature_1 >  16.55
|   |   |   |   |   |--- class: Chinstrap
|   |   |   |--- feature_1 >  16.70
|   |   |   |   |--- class: Adelie
|   |   |--- feature_0 >  42.40
|   |   |   |--- feature_2 <= 189.50
|   |   |   |   |--- class: Chinstrap
|   |   |   |--- feature_2 >  189.50
|   |   |   |   |--- class: Adelie
|   |--- feature_0 >  46.00
|   |   |--- class: Chinstrap
|--- feature_2 >  202.50
|   |--- feature_0 <= 55.05
|   |   |--- class: Gentoo
|   |--- feature_0 >  55.05
|   |   |--- class: Chinstrap
```

```
with open("decistion_tree_test.log", "w") as fout:
 fout.write(text_representation)
```

```
fig = plt.figure(figsize=(25,20))
tree.plot_tree(clf_test, feature_names=fn, class_names=cn,filled=True)
fig.savefig('imagename1.png')
```




```
fig.savefig("decistion_tree_test.png")
```

Visualize the Decision Tree on Overall Data

```
clf = DecisionTreeClassifier(random_state=1234)
dt = clf.fit(X, y)
```

```
text_representation = tree.export_text(clf)
print(text_representation)
```

```
|--- feature_2 <= 206.50
|   |--- feature_0 <= 43.35
|   |   |--- feature_0 <= 42.35
|   |   |   |--- feature_1 <= 16.65
|   |   |   |   |--- feature_0 <= 39.50
|   |   |   |   |   |--- class: Adelie
|   |   |   |   |--- feature_0 >  39.50
|   |   |   |   |   |--- class: Chinstrap
|   |   |   |--- feature_1 >  16.65
|   |   |   |   |--- class: Adelie
|   |   |--- feature_0 >  42.35
|   |   |   |--- feature_2 <= 189.50
|   |   |   |   |--- class: Chinstrap
|   |   |   |--- feature_2 >  189.50
|   |   |   |   |--- class: Adelie
|   |--- feature_0 >  43.35
```

```
|   |   |--- feature_3 <= 4125.00
|   |   |   |--- class: Chinstrap
|   |   |--- feature_3 >  4125.00
|   |   |   |--- feature_0 <= 48.80
|   |   |   |   |--- feature_3 <= 4612.50
|   |   |   |   |   |--- feature_3 <= 4175.00
|   |   |   |   |   |   |--- feature_2 <= 196.00
|   |   |   |   |   |   |   |--- class: Chinstrap
|   |   |   |   |   |   |--- feature_2 >  196.00
|   |   |   |   |   |   |   |--- class: Adelie
|   |   |   |   |   |--- feature_3 >  4175.00
|   |   |   |   |   |   |--- class: Adelie
|   |   |   |   |--- feature_3 >  4612.50
|   |   |   |   |   |--- class: Gentoo
|   |   |   |--- feature_0 >  48.80
|   |   |   |   |--- class: Chinstrap
|   |--- feature_2 >  206.50
|   |   |--- feature_1 <= 17.65
|   |   |   |--- class: Gentoo
|   |   |--- feature_1 >  17.65
|   |   |   |--- feature_0 <= 46.55
|   |   |   |   |--- class: Adelie
|   |   |   |--- feature_0 >  46.55
|   |   |   |   |--- class: Chinstrap
```

```
with open("decistion_tree.log", "w") as fout:
 fout.write(text_representation)
```

```
fig = plt.figure(figsize=(25,20))
tree.plot_tree(clf, feature_names=fn, class_names=cn,filled=True)
fig.savefig('imagename2.png')
```